

Drone Delivery Routing

Algorithm Design Manual

Alex Alvarado Pinto `alvaradopinto.a@northeastern.edu`
William Harnett `harnett.w@northeastern.edu`

1. Problem Domain

Our project falls under the **Graphs / Routing** domain. We model a drone delivery system as a shortest-path problem on a weighted graph, where the goal is to find the fastest round trip for a drone carrying two packages.

2. Problem Specification

Input:

- A map of locations modeled as an undirected weighted graph $G = (V, E)$, with $N = |V|$ locations and $M = |E|$ flight paths between them.
- A warehouse s — the starting and ending point for the drone.
- Two delivery destinations D_1 and D_2 , one package each.
- A distance $w(u, v) \geq 0$ for every flight path (u, v) .

Output: The shortest possible round trip $s \rightarrow \dots \rightarrow D_i \rightarrow \dots \rightarrow D_j \rightarrow \dots \rightarrow s$ that starts and ends at the warehouse and visits both destinations exactly once, along with its total distance.

Objective: Minimize the total flight distance of the drone's round trip. Because the drone flies at constant speed, minimizing distance is the same as minimizing total delivery time.

Assumptions:

- The drone flies at constant speed.
- All distances are non-negative (you cannot have a negative flight path).
- The map is fully connected — every location can be reached from the warehouse.
- The drone carries exactly 2 packages per trip and delivers both before returning.
- We do not model battery life, wind, or airspace restrictions — these are left for future work.

3. Algorithm Design

Approach: Dijkstra's algorithm (greedy shortest-path).

Because the drone carries exactly 2 packages, there are only $2! = 2$ possible delivery orders:

$$s \rightarrow D_1 \rightarrow D_2 \rightarrow s \quad \text{or} \quad s \rightarrow D_2 \rightarrow D_1 \rightarrow s.$$

The best route is whichever order has the smaller total distance. To find those distances, we run Dijkstra's algorithm twice — once starting from the warehouse s , and once starting from D_1 .

Those two runs give us every pairwise distance we need. We then plug the numbers into both cost formulas and return the cheaper route.

Why Dijkstra?

- It requires all edge weights to be non-negative — which holds here since distances cannot be negative.
- It gives exact shortest-path distances, so we can guarantee the route we return is truly optimal.
- With a binary min-heap it runs in $O((N + M) \log N)$, which is efficient for the map sizes we consider.

BUILD-GRAPH(*locs*, *edges*)

```

1  V = locs // all locations including warehouse
2  for each v ∈ V
3      adj[v] = [] // empty neighbor list
4  for each (u, v, c) ∈ edges // c is the edge weight
5      add (v, c) to adj[u]
6      add (u, c) to adj[v] // undirected: add both directions
7  return (V, adj)

```

DIJKSTRA(*G*, *s*, *w*)

```

1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  H = ∅
4  for each vertex u ∈ G.V
5      INSERT(H, u)
6  while H ≠ ∅
7      u = EXTRACT-MIN(H)
8      S = S ∪ {u}
9      for each vertex v ∈ G.Adj[u]
10         RELAX(u, v, w)
11         if the call of RELAX decreased v.d
12             DECREASE-KEY(H, v, v.d)
13 return (dist, prev)

```

INITIALIZE-SINGLE-SOURCE(*G*, *s*)

```

1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0

```

RELAX(u, v, w)

```
1  if  $v.d > u.d + w(u, v)$ 
2       $v.d = u.d + w(u, v)$ 
3       $v.\pi = u$ 
```

DELIVER-TWO(V, adj, s, D_1, D_2)

```
1  // Run Dijkstra from  $s$  and from  $D_1$ 
2   $(dist_s, prev_s) = \text{DIJKSTRA}(V, adj, s)$ 
3   $(dist_1, prev_1) = \text{DIJKSTRA}(V, adj, D_1)$ 
4  // Compare the two possible orderings
5   $cost_A = dist_s[D_1] + dist_1[D_2] + dist_s[D_2]$  //  $s \rightarrow D_1 \rightarrow D_2 \rightarrow s$ 
6   $cost_B = dist_s[D_2] + dist_1[D_2] + dist_s[D_1]$  //  $s \rightarrow D_2 \rightarrow D_1 \rightarrow s$ 
7  if  $cost_A \leq cost_B$ 
8       $first = D_1$ 
9       $second = D_2$ 
10 else  $first = D_2$ 
11      $second = D_1$ 
12 // Build the route:  $s \rightarrow first \rightarrow second \rightarrow s$ 
13  $leg_1 = \text{GET-PATH}(s, first, prev_s)$  // warehouse to first stop
14  $leg_2 = \text{GET-PATH}(D_1, D_2, prev_1)$  // between the two stops
15 if  $first = D_2$ 
16     REVERSE( $leg_2$ ) // flip  $D_1 \rightarrow D_2$  to get  $D_2 \rightarrow D_1$ 
17  $leg_3 = \text{REVERSE}(\text{GET-PATH}(s, second, prev_s))$  // second stop back to warehouse
18  $route = [s] \parallel leg_1 \parallel leg_2 \parallel leg_3$  // concatenate all legs
19 return ( $route, \min(cost_A, cost_B)$ )
```

GET-PATH($s, v, prev$)

```
1   $p = []$  // walk backwards from  $v$  to  $s$ 
2   $u = v$ 
3  while  $u \neq s$ 
4      put  $u$  at front of  $p$ 
5       $u = prev[u]$ 
6  return  $p$ 
```

4. Proof of Correctness

Throughout this section, $G = (V, E)$ is our undirected graph with non-negative edge weights w , and $\delta(a, b)$ denotes the true shortest-path distance between any two vertices a and b .

Part (a): Dijkstra gives the right distances

Goal: Show that when DIJKSTRA finishes, $dist[v] = \delta(s, v)$ for every vertex v .

We prove this by induction on $|S|$ — the number of vertices that have been pulled out of the heap so far. The property we maintain at every step is:

$$\text{Every vertex } u \in S \text{ satisfies } \text{dist}[u] = \delta(s, u).$$

Base case ($|S| = 1$). The first vertex pulled from the heap is always s , because INITIALIZE-SINGLE-SOURCE sets $s.d = 0$ and every other vertex to ∞ . So $\text{dist}[s] = 0 = \delta(s, s)$. The property holds.

Inductive hypothesis. Assume the property holds after the first $k \geq 1$ extractions: every vertex already in S has its correct shortest distance.

Inductive step (going from k to $k + 1$). Let u be the next vertex pulled from the heap. We show $\text{dist}[u] = \delta(s, u)$.

Upper bound. RELAX only ever updates $v.d$ to a value equal to the weight of a real path from s to v . So $\text{dist}[u]$ can never drop below the true shortest distance: $\text{dist}[u] \geq \delta(s, u)$.

Lower bound (by contradiction). Suppose $\text{dist}[u] > \delta(s, u)$. Then there exists a cheaper path P from s to u that we have not yet accounted for. Since $s \in S$ and $u \notin S$, path P must cross from inside S to outside S at some point. Call the first such crossing edge (x, y) , where $x \in S$ and $y \notin S$.

1. By the inductive hypothesis, $\text{dist}[x] = \delta(s, x)$.
2. When x was extracted, edge (x, y) was relaxed, so:

$$\text{dist}[y] \leq \text{dist}[x] + w(x, y) = \delta(s, x) + w(x, y) = \delta(s, y).$$

The last equality holds because the subpath of P from s to y is itself a shortest path.

3. Since y is still in the heap and u was chosen as the minimum, $\text{dist}[u] \leq \text{dist}[y]$.
4. All weights are non-negative, so $\delta(s, y) \leq \delta(s, u)$.
5. Chaining steps 2–4: $\text{dist}[u] \leq \text{dist}[y] \leq \delta(s, y) \leq \delta(s, u)$, contradicting $\text{dist}[u] > \delta(s, u)$.

Therefore $\text{dist}[u] = \delta(s, u)$. By induction, every vertex has its correct distance when the algorithm terminates.

Part (b): The prev pointers trace a real shortest path

Goal: Show that following the *prev* pointers from any vertex v back to s reconstructs an actual shortest path.

We prove this by induction on the number of edges in the path.

Base case. If $v = s$, the path has zero edges and total weight $\delta(s, s) = 0$. Nothing to prove.

Inductive step. Let $v \neq s$ and $u = \text{prev}[v]$. The pointer $\text{prev}[v]$ is set to u inside RELAX only when a shorter path through u is found. By Part (a), at termination $\text{dist}[v] = \delta(s, v)$ and $\text{dist}[u] = \delta(s, u)$, so the final relaxation gives:

$$\delta(s, v) = \delta(s, u) + w(u, v).$$

By the inductive hypothesis, following *prev* from u to s yields a path of weight $\delta(s, u)$. Appending edge (u, v) gives a path of weight $\delta(s, u) + w(u, v) = \delta(s, v)$, which is a shortest path.

Reversal is also valid. Because G is undirected, every edge works in both directions. If $P = \langle v_0, v_1, \dots, v_k \rangle$ is a shortest path from v_0 to v_k , then its reverse $\langle v_k, \dots, v_0 \rangle$ is a valid path from v_k to v_0 with the same weight. Since $\delta(v_0, v_k) = \delta(v_k, v_0)$ in an undirected graph, the reversed path is also shortest. This justifies every REVERSE call in DELIVER-TWO.

Part (c): Deliver-Two picks the best route

Goal: Show that DELIVER-TWO returns the minimum-cost round trip visiting both D_1 and D_2 .

All orderings are checked. Any round trip from s through D_1 and D_2 must visit one of them first. There are exactly $2! = 2$ such orderings. The algorithm computes the cost of both, so the optimal one cannot be missed.

Each cost is correct. For a given ordering, the cheapest route takes a shortest path on each of the three legs. By Part (a), the two Dijkstra runs give exact distances. Because G is undirected, $\delta(D_2, D_1) = \delta(D_1, D_2)$ and $\delta(D_i, s) = \delta(s, D_i)$, so two runs are enough. Substituting:

$$\begin{aligned} cost_A &= \delta(s, D_1) + \delta(D_1, D_2) + \delta(D_2, s), \\ cost_B &= \delta(s, D_2) + \delta(D_2, D_1) + \delta(D_1, s). \end{aligned}$$

Both values are exact, so $\min(cost_A, cost_B)$ is the global minimum.

The reconstructed path is valid. Each leg is built with GET-PATH, which Part (b) guarantees traces a true shortest path. When the ordering requires $D_2 \rightarrow D_1$, reversing leg_2 is valid by Part (b). The return leg is the reverse of a shortest path from s to $second$, also valid. Concatenating the three legs produces a valid round trip whose total weight equals $\min(cost_A, cost_B)$. □

5. Run-Time Analysis

We analyze the **worst-case** runtime, which occurs when the graph is dense and every edge visit triggers a heap update.

Because Dijkstra is iterative (not recursive), there is no recurrence to solve. We analyze the runtime by counting operations directly.

Cost of one call to DIJKSTRA (N vertices, M edges, binary min-heap):

- **Initialization.** Set $v.d = \infty$ and $v.\pi = \text{NIL}$ for all N vertices. Cost: $\Theta(N)$.
- **Building the heap (lines 4–5).** Each vertex is inserted once. Inserting into a binary min-heap costs $O(\log N)$ because the heap has at most $\log N$ levels. Cost: $N \times O(\log N) = \Theta(N \log N)$.
- **Extract-Min (line 7).** Each vertex is removed from the heap exactly once, also at $O(\log N)$ each. Cost: $N \times O(\log N) = \Theta(N \log N)$.
- **Relax and Decrease-Key (lines 10–12).** Because G is undirected, each edge $\{u, v\}$ appears in both adjacency lists, so the inner loop executes at most $2M$ times in total. Each RELAX is $O(1)$; each DECREASE-KEY is $O(\log N)$. Cost: $2M \times O(\log N) = O(M \log N)$.

Summing all terms:

$$\Theta(N) + \Theta(N \log N) + \Theta(N \log N) + O(M \log N) = \Theta((N + M) \log N).$$

Total cost of DELIVER-TWO. We run DIJKSTRA exactly twice:

$$2 \times \Theta((N + M) \log N) = \Theta((N + M) \log N).$$

All other steps (comparing costs, reconstructing paths, reversing, and concatenating) are $O(N)$ and are dominated by the Dijkstra runs.

The overall worst-case runtime is:

$$\boxed{\Theta((N + M) \log N)}$$

6. Modeling and Fairness Analysis

6.1 What is the algorithm optimizing for?

The algorithm minimizes the total flight distance of the drone’s round trip from the warehouse to D_1 and D_2 and back. Because the drone flies at constant speed, shorter distance means faster delivery.

Who benefits: Delivery companies benefit most directly; shorter routes mean lower energy costs, less wear on the drone, and higher throughput. Customers at both destinations benefit from faster deliveries. Drone operators and developers benefit from simpler, more predictable flight paths.

Who might not: The algorithm treats both deliveries as points on a map and nothing more. A customer who lives far from the warehouse will almost always be visited last, regardless of whether their package is more urgent. If delivery zones align with income or population density — for example, if wealthier neighborhoods cluster near the warehouse while lower-income neighborhoods are farther out — the optimization can end up systematically favoring some customers over others without intending to.

6.2 Adversarial Input

Consider a simple three-location example:

Warehouse s

D_1 : nearby, distance 1 from s

D_2 : far away, distance 100 from s , distance 99 from D_1

Computing both orderings:

$$\begin{aligned} cost_A &= 1 + 99 + 100 = 200 & (s \rightarrow D_1 \rightarrow D_2 \rightarrow s), \\ cost_B &= 100 + 99 + 1 = 200 & (s \rightarrow D_2 \rightarrow D_1 \rightarrow s). \end{aligned}$$

Both orderings cost the same, so the algorithm picks arbitrarily — whichever satisfies $cost_A \leq cost_B$. No matter which is chosen, D_2 still experiences a long wait. If that package contains medical supplies or something time-sensitive, the algorithm has no way to prioritize it. The result is distance-optimal but potentially unacceptable in practice.

6.3 Fairness Constraint

We introduce a **deadline constraint**: each destination D_i has a maximum acceptable wait time T_i , measured in minutes from the moment the drone departs. Since the drone flies at constant speed v , the time to reach a stop is its distance divided by v . A route is only valid if:

$$\frac{\delta(s, D_i)}{v} \leq T_i \quad \text{for every stop } D_i.$$

If the distance-optimal ordering would cause a customer to miss their deadline, the algorithm must pick the other ordering instead — even if it costs more distance. If neither ordering meets both deadlines, the algorithm reports that the two deliveries cannot share a single trip.

6.4 Algorithmic Impact

Does the original algorithm still work? Mostly yes. The two DIJKSTRA runs and the cost computations are completely unchanged. The only addition is a feasibility check before selecting a route: for each ordering, compute the arrival time at each stop and verify it is within the deadline. If both pass, pick the cheaper one as before. If only one passes, take that one. If neither passes, report no valid trip exists.

Does the runtime change? No. The feasibility check computes two arrival times per ordering — that is $O(1)$ extra work. The dominant cost remains the two Dijkstra calls, so the overall worst-case runtime is still:

$$\Theta((N + M) \log N).$$